FIT3155: Week 5 Lab Sheet

(Questions are based on Week 4 topic, Ukkonen's algorithm for suffix tree construction.)

Objectives: Develop a clear understanding of Ukkonen's algorithm of a linear-time suffix tree construction and its common applications.

Conceptual exercises

1. For a string $S[1\dots n]$, reason the following without getting bogged down in implementational details of the Ukkonen's algorithm):

   (a) Conceptually, what information are you storing at the end of each phase of this algorithm?

   ### Ans: 1(a)

   In each phase, the Ukkonen's algorithm builds a growing implicit suffix tree that stores information of all suffixes of a growing prefix of S. Specifically, at the end of any phase i ($\forall 1 \le i \le n$), the implicit suffix tree will hold information of all i suffixes of the form $S[j\dots i]$ ($\forall 1 \le j \le i$) of the prefix $S[1\dots i]$.

   (b) Going from some phase i to the next phase $i+1$, when extending $\alpha = S[j\dots i]$ (i.e., the suffix j of phase i) by one more character (say) $x = S[i+1]$, reason abstractly how (or what aspects of) the implicit suffix tree changes upon encountering the following suffix-extension scenarios:

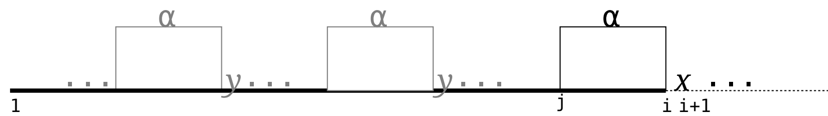
   - α is encountered for the **first time** starting at position j and never at any $j' < j$.
     (This will aid your understanding of Rule 1 suffix extension.)
     As an example, handling the following string
     `1 2 3 4 5 6 7 8 9 10`
     `a b c c b c c b d d ...,`
     this scenario presents itself for extension $j = 6$ in phase $i + 1 = 10$.

     ### Ans: 1(b) Rule 1

     Performing extension j of phase i, if $\alpha = S[j\dots i]$ was never previously observed at any position $j' < j$, then the path from the root following $S[j\dots i]$ is unique. By construction, this path will have to terminate at a leaf node labeled with suffix index j, where the edge to that leaf ends at position i.

     In phase $i+1$, extending the previous suffix $S[j\dots i]$ by $x = S[i+1]$, the leaf's edge label increments to $i+1$.[*]

[*] In Ukkonen's algorithm, this increment happens automatically via the `global_end` trick. Rule 1 is never explicitly applied.



- α was previously observed (one or more times before j), but always followed by some same character $y \neq x$.
(This will aid your understanding of Rule 2 (regular) suffix extension.)

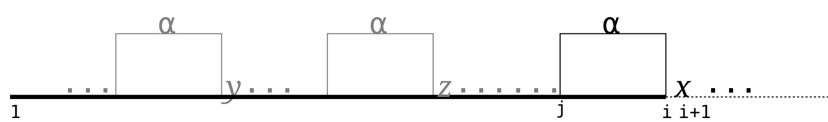
As an example, handling the following string

```
1 2 3 4 5 6 7 8 9 10 11 12 13
a b c c b c c b d d  c  c  d  ...,
```

this scenario presents itself for extension $j = 11$ in phase $i + 1 = 13$.

Ans: 1(b) Rule 2 regular

Before extension j of phase $i+1$, suppose $\alpha = S[j \dots i]$ has occurred $k \geq 1$ times at positions $\{j'_p \mid 1 \leq p \leq k\} < j$. Further, the condition about the characters following all previous occurrences of α implies a set with a singleton character $y = \{y_p = S[j'_p + |\alpha|] \mid 1 \leq p \leq k\} \neq x$.
Then, in the implicit suffix tree, the path from the root along $\alpha y \dots$ will be shared by k suffixes. By construction, there cannot be any internal node between the path α and, the character that follows, y, because every internal node in a suffix tree has an arity (number of outgoing edges) of at least 2. If there were an existing internal node between α and y, it would imply the existence of another occurrence of α at some position $< j$ that is followed by a character different from y, which contradicts the scenario being considered.
In phase $i + 1$, since α is being extended by $x = S[i + 1] \neq y$, the suffix extension j requires the implicit suffix tree to accommodate a new path extending α along character x. Consequently, a new internal node is created at the end of the path α in the tree (where none previously existed), along with a new leaf node (storing suffix index j) branching off along x from this internal node. The edge leading to the new leaf will have the label $[i + 1, \texttt{global\_end}]$ (as a proxy for $[i + 1, i + 1]$).



- α was previously observed (two or more times before j), and there were at least two distinct characters following those occurrences, each **different** from x.
(This will aid your understanding of Rule 2 (alternate) suffix extension.)

As an example, handling the following string

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
a b c c b c c b d d  c  c  d  c  c  b  d  c  c  b  c  c  c  b  a  ...,
```

this scenario presents itself for extension $j = 22$ in phase $i + 1 = 25$.

Ans: 1(b) Rule 2 alternate

Before extension j of phase $i+1$, suppose $\alpha = S[j \dots i]$ has occurred $k \geq 2$ times at positions $\{j'_1, j'_2, \dots, j'_k\} < j$. Further, the condition about the characters following previous occurrences of α implies there exists a set of characters of size ≥ 2, where $x \notin \{y_p = S[j'_p + |\alpha|] \mid 1 \leq p \leq k\}$.
Then, in the implicit suffix tree before the extension, $k \geq 2$ suffixes will share a common path from the root α. There will have to be an internal node (say) u at the end of α, from which $k \geq 2$ edges branch out along the distinct characters in the set $\{y_p\}$.

In phase $i + 1$, since α is being extended by $x = S[i + 1]$ and $x \notin \{y_p\}$, extension j requires a new branch from this existing internal node u extending α by x. Consequently, a new leaf node (storing suffix index j) branches off u, with edge label $[i + 1, i + 1]$ or rather $[i + 1, \texttt{global\_end}]$.

- α followed by x was previously observed at some position $j' < j$? (This will aid your understanding of Rule 3 suffix extension.)

  As an example, handling the following string

  1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
  a b c c b c c b d d c c d c c b d c c b c c c b a c c b d ...,

  this scenario presents itself for extension $j = 26$ in phase $i + 1 = 29$.

Ans: 1(b) Rule 3

Before extension j of phase $i+1$, suppose $\alpha = S[j \dots i]$ has occurred $k \geq 1$ times at positions $\{j'_1, j'_2, \dots, j'_k\} < j$. Further, the condition about the characters following previous occurrences of α implies there exists a set of characters of size ≥ 1, where $x \in \{y_p = S[j'_p + |\alpha|] \mid 1 \leq p \leq k\}$.

In phase $i+1$, since α is being extended by $x = S[i+1]$ and $x \in \{y_p\}$, then the information of the suffix starting at j is already within the current state of the implicit suffix tree, requiring no further modifications to the tree.

This is Rule 3. In fact, applying trick 3 of the Ukkonen's algorithm, future extensions $\{j+1, j+2, \dots i+1\}$ of any phase $i+1$ become unnecessary after recognizing Rule 3 in some extension j of that phase, as they will all be Rule 3 and nothing changes in the tree when Rule 3 is observed.

2. For a suffix extension j in phase i, suppose the applied rule is one of: Rule 1, Rule 2 (regular), Rule 2 (alternate), or Rule 3.

   For each case determine which rules are possible for:

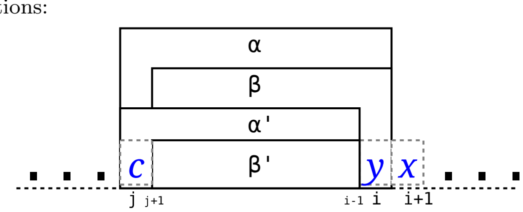
   (a) the same extension j in phase $i+1$;

   (b) the next extension $j+1$ in phase i.

   Note: Reasoning this question, among other things, will lay a groundwork for understanding Tricks 3 and 4 used in Ukkonen's algorithm (Suggestion: It will help to draw abstract string diagrams like those in this lab sheet, to guide your reasoning for this question.)

Ans: 2

We will use the following notations:
$\alpha' = S[j \dots i-1]$,
$\alpha = S[j \dots i] = \alpha' y$,
$\beta' = S[j+1 \dots i-1]$,
$\beta = S[j+1 \dots i] = \beta' y$,
$S[j] = c$,
$S[i] = y$,
$S[i+1] = x$.



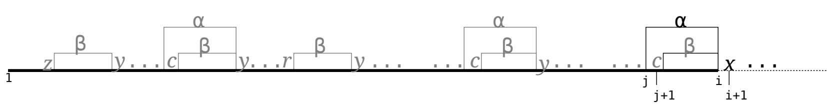
Extension j of Phase i		**Extension j of phase $i+1$**
Rule 1: This implies α' was never previously observed at any position $j' < j$. Hence, $\alpha = \alpha' y$ also could not have been observed at any $j' < j$, so the path α from the root in the tree, has to end in a leaf node by construction.	$\Longrightarrow$	***Rule 1:*** Since α was never previously observed, the current extension αx also would not be observed at any $j' < j$. Thus, the αx path from the root ends in a leaf node labeled j, making this a Rule 1 extension, and rules out all other rules.
Rule 2 (regular or alternate): If extension j of phase i yields Rule 2 regular or alternate, this implies α' branches (either via a new internal node or via an existing one) along character y ending in a new leaf (with suffix index j).	$\Longrightarrow$	***Rule 1:*** Given $\alpha = \alpha' y$ was a new branching path in the same extension of the previous phase (due to application of Rule 2 regular or alternate), α ending in a leaf node, by construction, would not have been observed at any $j' < j$. It then follows that extending this further to αx will be a leaf extension, making it Rule 1, while ruling out all other rules.

Note: The above two implications form the basis of Trick 4 in Ukkonen's algorithm. By tracking $\texttt{Last}_{j_i}$ (the extension where Rule 1 or Rule 2 was **last** observed in phase i), all suffix extensions $j = \{1, \dots, \texttt{Last}_{j_i}\}$ in phase $i+1$ become Rule 1 extensions. Thus, using the $\texttt{global\_end}$ trick, these Rule 1 extensions become automatic/implicit, when the phase index increments from $i \to i+1$, requiring $O(1)$ effort.

...cont'd. **Extension j of Phase i**		..cont'd. **Extension j of phase $i+1$**
Rule 3: If extension j of phase i yields Rule 3, this means there exists at least one previous occurrence of α' at some $j' < j$ that was followed by y.	$\Longrightarrow$	***Rule 2 (regular or alternate) or Rule 3:*** **Rule 1** Impossible as $\alpha = \alpha' y$ was previously observed at some $j' < j$. This implies there is a longer suffix path of the form $\alpha \ldots \alpha$ already in the tree. Thus αx starting at j shares a common prefix path (α) with at least one longer suffix starting at j', which rules out αx from being Rule 1. **Rule 2 regular** If all previous α was always followed by the same character (say) $z \neq x$. **Rule 2 alternate** If two or more previous αs were followed by at least two distinct characters $(p, q) \neq x$. **Rule 3** If one or more previous α was followed by x.

Extension j of Phase i		**Extension $j+1$ of Phase i**
Rule 1: This implies α' was never previously observed at any position $j' < j$. Hence, $\alpha = \alpha' y$ also could not have been observed at any $j' < j$, so its path ends in a leaf node.	$\Longrightarrow$	***One of Rules 1, 2 (regular/alternate), or 3:*** In extension $j+1$ of phase i, β' needs to be extended to $\beta = \beta' y$. The uniqueness of $\alpha' = c\beta'$ does not constrain any earlier β' occurrences $< j$. So, β' can appear zero or more times previously, independently. Thus: **Rule 1:** If β' was never observed at any $j' < j$. **Rule 2 regular:** If β' observed ≥ 1 times, always followed by some same character (say) $z \neq y$. **Rule 2 alternate:** If β' observed ≥ 2 times, with ≥ 2 distinct characters (say) $p, q \neq y$. **Rule 3:** If β' observed ≥ 1 times, at least one followed by y.
Rule 2 (regular): If extension j of phase i yields Rule 2 regular, this implies α' branches via a new internal node created after the path α' from the root, along the phase's character y ending in a new leaf (with suffix index j).	$\Longrightarrow$	***Rule 2 (regular/alternate), or 3:*** In extension $j+1$ of phase i, β' needs to be extended to $\beta = \beta' y$. Since, Rule 2 (regular) was observed in the previous extension, $\alpha' = c\beta'$ would have been observed previously one or more time $< j$. This implies β' would have also appeared (as part of α') at least as many times as α', and all those instances will have be followed by the same letter (say) $z \neq y$. In addition to this, β' can also appear independently of α' without any constrains on what character follows such instances. Based on this background: **Rule 1** Impossible. There are longer suffix(es) starting at $j' < j$ that describe paths from the root that contain β' as a proper prefix. Thus, the extension $\beta' y$ can never end leading into a leaf node. **Rule 2 regular:** If those β'(s) observed independently of α'(s) were also followed by z, a new branching path has to be created after β' in the direction of y after creating a **new internal node** and hanging a new leaf node (with suffix index $j+1$) off it. **Rule 2 alternate:** Differing from above, if those β' observed independently of α' were followed by characters $\neq z, y$, then there will already exist a path from root to an **existing internal node** describing β'. A new branching path is created in the direction of y hanging a new leaf node (suffix index $j+1$) off it. **Rule 3:** Differing from above, if one of those β' observed independently of α' was followed by the character y, then there is a path from root to an existing internal node that will have a branch towards (at least) $y \ldots$ and $z \ldots$. In the current extension, since the branching path towards $y \ldots$ already exists, there is nothing more to do.
Note: The implications form the basis of resolving suffix links in Ukkonen's algorithm, whenever a new internal node s is created. The implication of Rule 2 (regular) in the next extension implies a new internal node t is created that receives the suffix link from s. The implication of Rule 2 (alternate) or Rule 3 in the next extension implies there is already an existing internal node t that receives the suffix link from s.		

..cont'd. **Extension j of Phase i**		...cont'd. **Extension $j+1$ of Phase i**
Rule 2 (alternate): If extension j of phase i yields Rule 2 alternate, this implies α' branches via an existing internal node created after the path α' from the root, along the phase's character y ending in a new leaf (with suffix index j).	$\Longrightarrow$	***One of Rules 1, 2 (regular/alternate), or 3:*** In extension $j+1$ of phase i, β' needs to be extended to $\beta = \beta' y$. Since Rule 2 (alternate) was observed in the previous extension, $\alpha' = c\beta'$ would have been observed previously two or more time $< j$ with at least two distinct characters (say) $\neq y$. This implies β' would have also appeared (as part of α') with those distinct characters $\neq y$ following β' instances. In addition to this, β' can also appear independently of α' without any constrains on what character follows those instances. Based on this background: **Rule 1** Impossible. There are longer suffix(es) starting at $j' < j$ that describe paths from the root that contain β' as a proper prefix. Thus, the extension $\beta' y$ can never end leading into a leaf node. **Rule 2 (regular)** Impossible. Since Rule 2 alternate in the previous extension implies α' branches into at least two distinct paths, it follows that β' should also branch into at least two distinct paths, ruling out any necessity to create a new internal node to account for the incoming y character of the phase. Only possibilities are those below. **Rule 2 alternate:** If no previous observation of independent β' followed by y. **Rule 3:** If previous observation of independent β' followed by y.
Rule 3: If extension j of phase i yields Rule 3, this implies the prior existence of $\alpha' y \ldots$ path from the root in the implicit suffix tree before this extension was undertaken.	$\Longrightarrow$	***Rule 3 only:*** In extension $j+1$ of phase i, β' needs to be extended to $\beta = \beta' y$. Since Rule 3 was observed in the previous extension, $\alpha' y = c\beta' y$ would have been observed previously one or more time $< j$. This implies $\beta' y$ already exists, making this and all subsequent extensions necessarily Rule 3. Note: This implication forms the basis of the phase show-stopper rule (trick 3) of Ukkonen's algorithm.

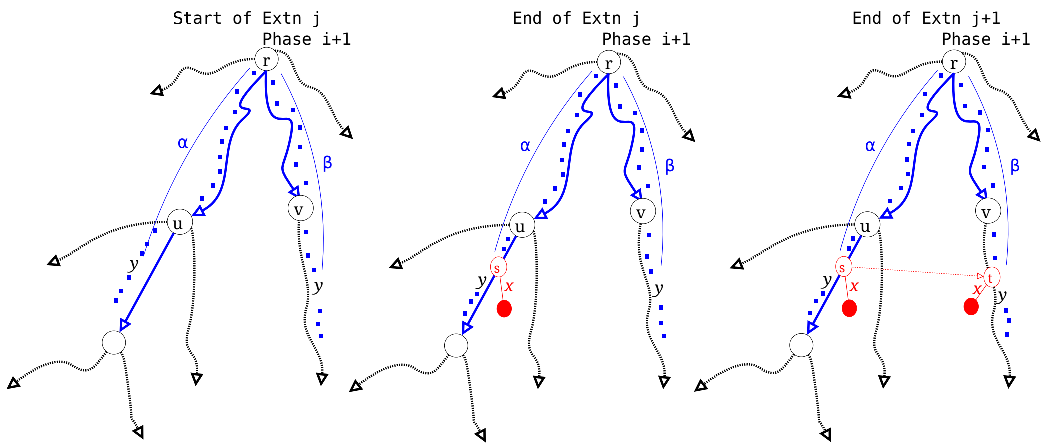


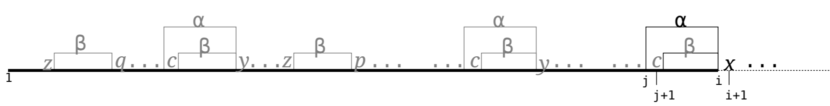
3. For a string $S[1 \ldots n]$, let $\alpha = S[j \ldots i]$ and $x = S[i+1]$. Write α as $\alpha = c\beta$, where $c = S[j]$ and $\beta = S[j+1 \ldots i]$.

Suppose that both substrings α and β (with β also occurring independently of α, i.e., preceded by a different character) have previously appeared in the prefix $S[1 \ldots i-1]$, as illustrated above. (Concrete example:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
b b a b a c b a b a b a a a c b a b a a b a b a c b a a b a b c ...,
Explore suffix extensions $j = 28$ followed by $j+1 = 29$ in phase $i+1 = 32$.)

- At the start of suffix extension j in phase $i+1$, sketch abstractly how the subtree after the path labeled α will look like. Do the same for β.
- How does this subtree change by the end of suffix extension j of phase $i+1$?
- How does it further change by the end of suffix extension $j+1$ of phase $i+1$?
- Is there a need to create a suffix link in this situation? If so, identify in your sketch the source and target node of this suffix link?

Ans: 3



4. 

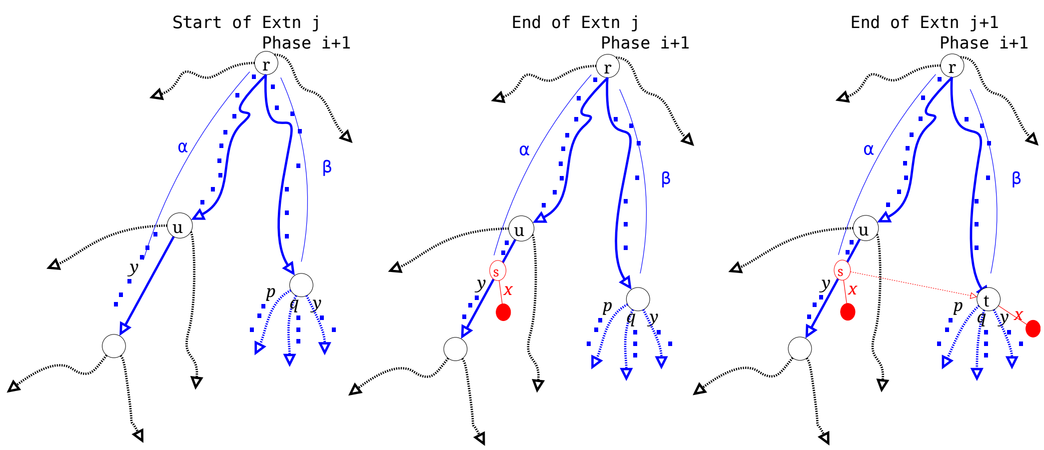
Redo the above questions for this abstract characterization of the string.

(Note: slightly modified example string below, compared to the one in the previous question:

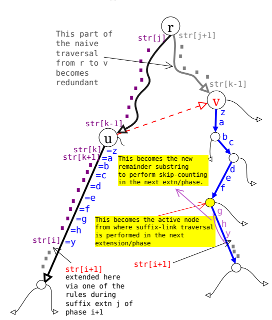
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
b b a b b c b a b a b a a a c b a b d a b a b a c b a a b a b c ...,

Explore suffix extensions $j = 28$ followed by $j + 1 = 29$ in phase $i + 1 = 32$.)

Ans: 4



5. Consider the following (abstract) illustration of the *skip-count* trick:



Use it to understand the definitions of `Active node`, `Remainder` and `suffix link` and how they factor into the skip-counting traversal. Understand how (potentially) the active node and remainder changes after each extension or after each phase.

Ans: 5

This is a revision question. Refer to the lecture slides/notes.

6. (Self-study question:) Manually trace out the mechanics of Ukkonen's algorithm (with tricks applied) for the string $S = \overset{1}{\texttt{a}}\ \overset{2}{\texttt{b}}\ \overset{3}{\texttt{a}}\ \overset{4}{\texttt{a}}\ \overset{5}{\texttt{b}}\ \overset{6}{\texttt{a}}\ \overset{7}{\texttt{\$}}$.

Ans: 6

See appended slides.

7. (Self-study question:) Reason the worst-case space-complexity required to store the entire suffix tree of any string $S[1 \dots n]$ derived from some alphabet Σ.

Ans: 7

For a suffix tree of any string $S[1 \dots n]$, to characterize the upper bound of the space required to for its suffix tree, we will bound:

(a) the total space required to store all the nodes of the suffix tree:

- The (explicit) suffix tree of S has exactly n leaf nodes (as every suffix will have a unique terminal character that forces branching towards that character).
- The number of internal nodes is at most $n - 1$. Why? This is because the number of internal nodes for any possible tree with n leaves is maximized when each internal node has the fewest number of children. In a suffix tree, every internal node has ≥ 2 children, so the maximum possible is when every internal node has exactly $= 2$ children (that is, the tree is a strict binary tree). A strict binary tree with n leaves has exactly $n-1$ internal nodes, therefore, no suffix tree with n leaves can have more than $n - 1$ internal nodes.
- Overall, there are n leaf nodes and at most $n-1$ internal nodes, totaling $2n-1$ nodes. Each internal node stores pointers to children (and the number of such pointers in the worst case is the size of the alphabet size Σ, practically can be considered a constant) and a suffix link. A leaf node just stores a suffix index (also requiring constant space).
- Thus, the space over all nodes is $(2n - 1) \times O(1) = O(n)$.

(b) the total space required to store all the edges of the suffix tree:

- Any (general) tree with m nodes will have $m - 1$ edges – this is a property of any general tree.
- We showed above that there are at most $2n - 1$ nodes (internal+leaf) in a suffix tree, yielding $2n - 2$ edges in the worst case.
- Each edge stores a pair of indexes $[sp, ep]$ that refer to the substring start and end points in S, requiring constant space.
- Hence, total space to store all edges is $(2n - 2) \times O(1) = O(n)$.

8. (Self-study questions:) Reason how the following algorithmic problems can be solved after constructing an (explicit) suffix tree of a given string $S[1 \dots n]$:

(a) Given another string $t[1 \dots m \leq n]$, find all positions (if any) where t appears in S. What is its time complexity?

Ans: 8a

Traverse from the root of the suffix tree of S, matching $t[1], t[2], \dots$ character-by-character until either:

- some character $t[k]$ $(1 \leq k \leq m)$ does not exist on any edge along the path, or
- the entire pattern $t[1 \dots m]$ is successfully matched.

If $t[1 \dots m]$ is found, all leaf nodes hanging below this path correspond to suffixes of S that contain t as a prefix. The suffix indices at these leaves give the starting positions where t occurs in S.

Time complexity will be $O(m + \texttt{multiplicity})$

- Effort for traversal is clearly $O(m)$ as each character of t is being search in the worst-case.
- Effort to report the positions: $O(\texttt{multiplicity})$, as you will have to traverse the entire subtree below the path $t[1 \dots m]$ (when it exists) to get the suffix index information from the leaf nodes hanging below the searched path.

(b) Find the suffix array of string S. What is its time complexity?

Ans: 8b

Perform a lexicographic traversal from the root node of the suffix tree of S. This will visit the leaf nodes storingsuffix indexes in a lexicogrpahic order, which forms the suffix array.

Time complexity will be $O(n)$ as, in any traversal of a tree, each edge is visited twice (once going down and once climbing back up).

(c) Find the BWT of string S. What is its time complexity?

Ans: 8c

Recall from Week 3, that letters in S that (cyclically) precedes corresponding positions stored in suffix array gives the string's BWT.

Time complexity will be $O(n)$ as, computing a suffix array via traversal of a suffix tree is $O(n)$, and converting eah suffix position in the suffix array to the corresponding letter in the BWT requires $O(1)$ effort.